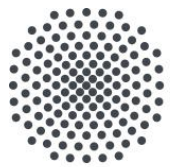


Low-Code Quantum Algorithm Modeling and Execution for Hybrid Cloud Environments



University of Stuttgart

**Lavinia Stiliadou, Johanna Barzen,
Fabian Bühler, Daniel Georg, Sharon-Naemi Stiliadou**

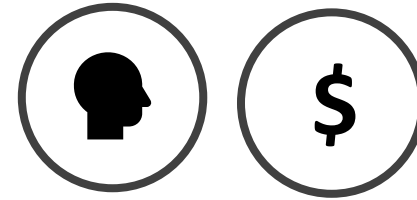
stiliadou@iaas.uni-stuttgart.de

Institute of Architecture of Application Systems

Motivation

- Classical Bit

- like a coin, either 0 (head) or 1 (tail)



- Qubit

- can be both tails and heads at the same time



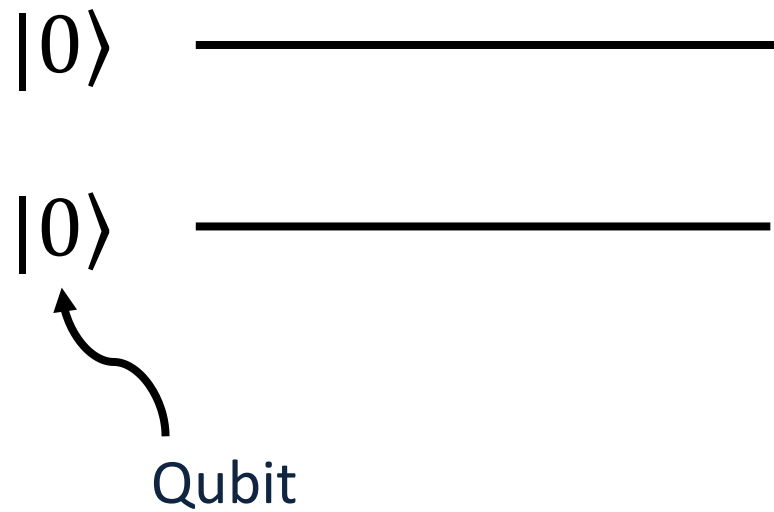
- Superposition

- many states at the same time

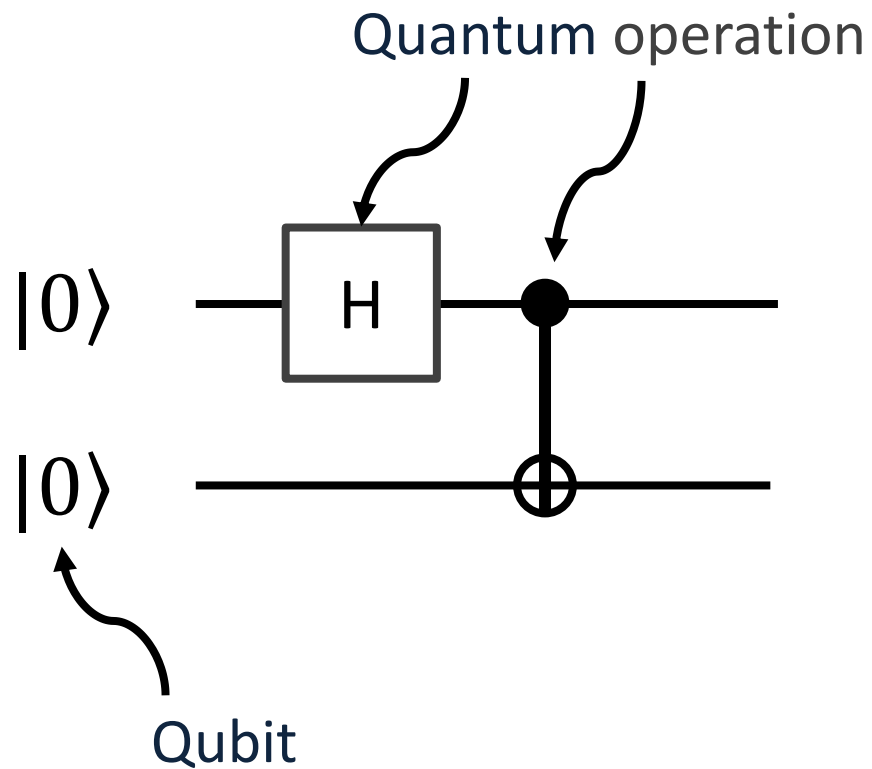
- Measurement

- forces the qubits to collapse into a definite classical value

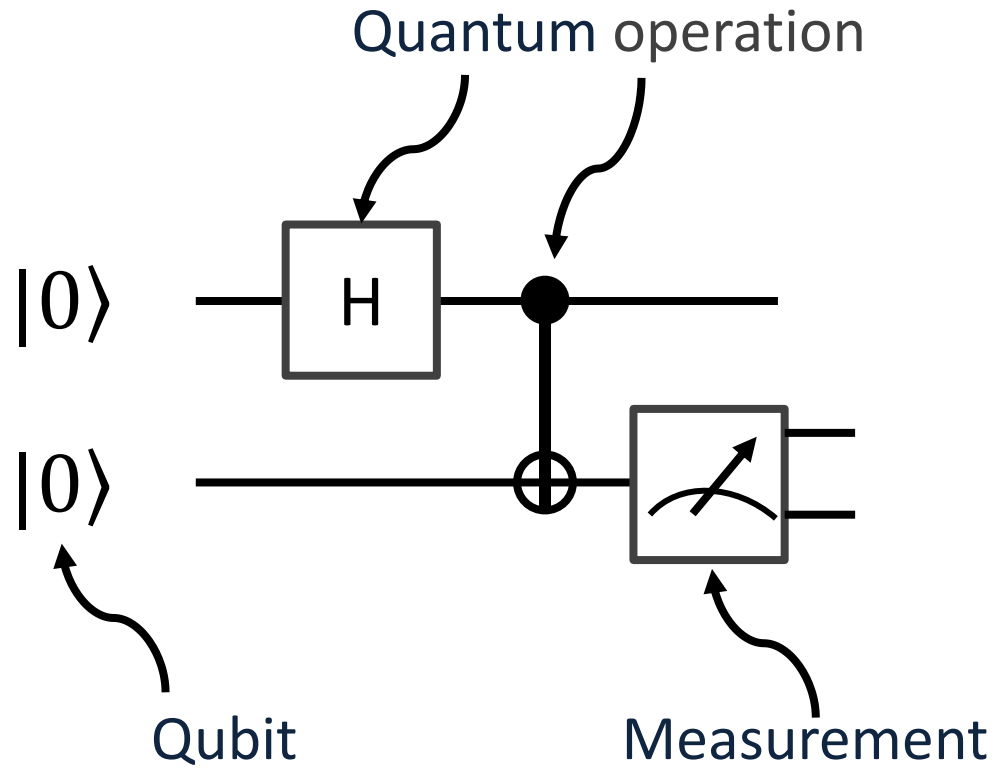
Motivation



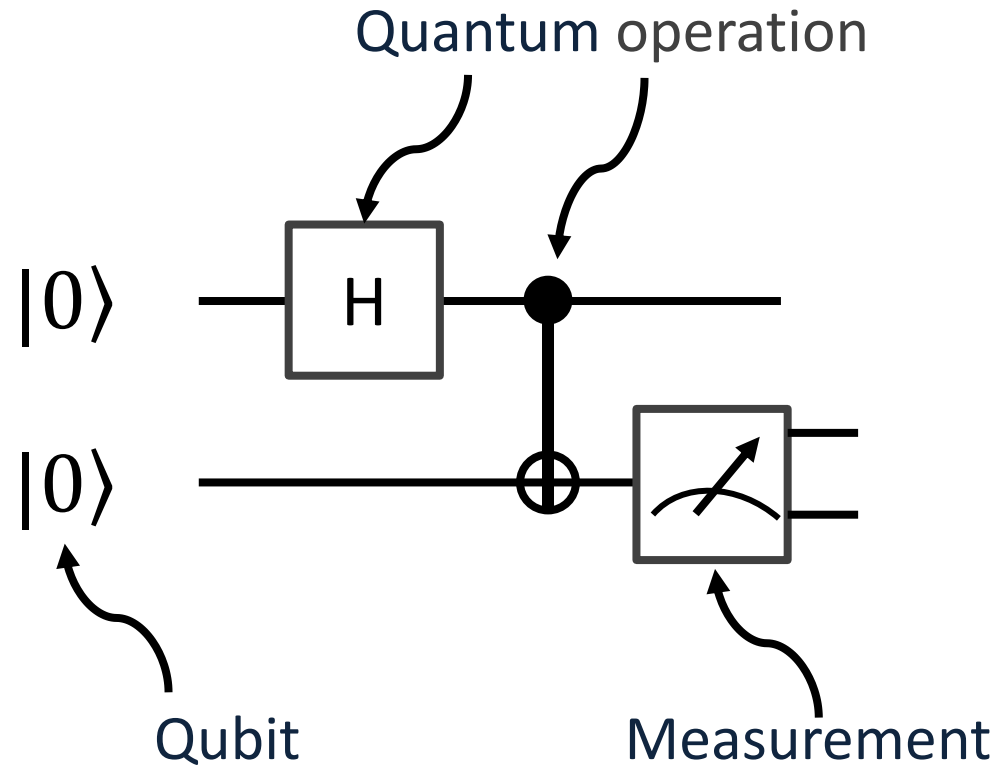
Motivation



Motivation



Motivation



```
// Imports
```

```
qc = QuantumCircuit(2, 1)
```

```
qc.h(0)
```

```
qc.cx(0, 1)
```

```
qc.measure(0, 0)
```

Motivation

- Existing quantum low-code platforms:
 - Often closed-source
 - Vendor-locked
 - Circuit-level only

Research Question

- To make quantum computing accessible:
 - Users need visual abstractions instead of low-level coding
 - Hybrid quantum-classical operations must be represented intuitively
 - Different expertise levels must be supported
 - Models should remain reusable and extensible

Research Question:

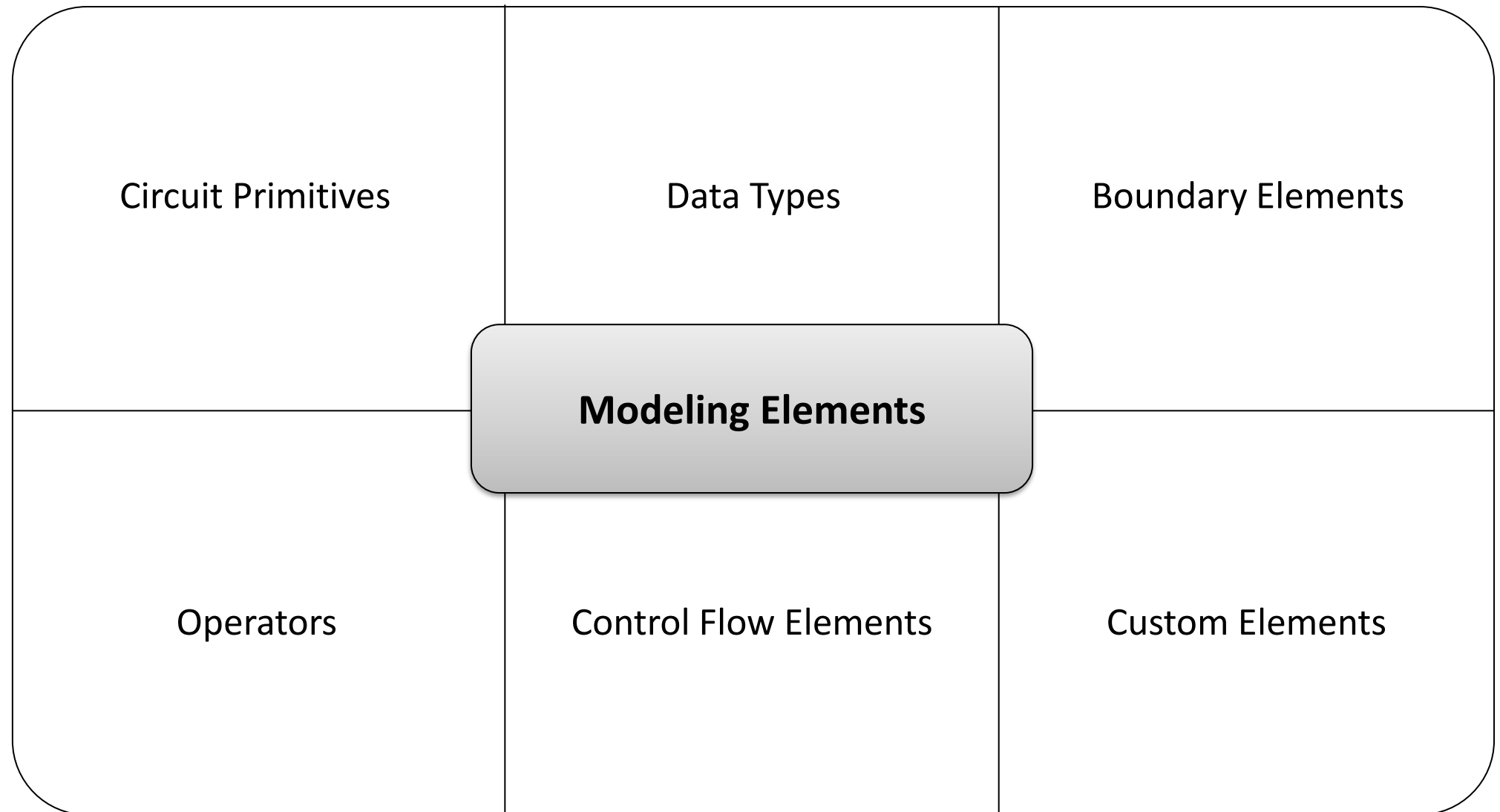
Which **abstractions** and **modeling elements** are required to support modeling of quantum algorithms in a **low-code platform**?

Modeling Language

- Icon
 - Basis Encoding Pattern Icon
- Type
 - State Preparation
- Properties
 - Encoding Type
 - Bound
 - Implementation (optional)
- Inputs
 - Value
- Outputs
 - Quantum Register

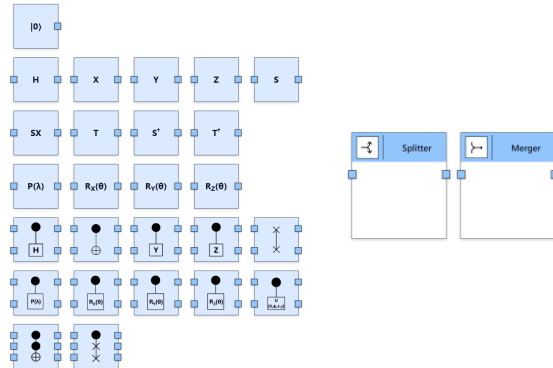
The image shows a software interface for configuring a quantum register. It features a blue header bar with a small icon and the title 'Encode Value'. Below the header, there are two dropdown menus: 'Encoding Type' (set to 'Basis Encoding') and 'Bound' (with a placeholder 'Enter bound'). A yellow callout box labeled 'Value' with 'type: any' is positioned to the left of the main content area. In the bottom right corner, a light blue box labeled 'Output: type: quantum register' contains two input fields: 'Identifier' (a solid blue border) and 'Size' (a dashed blue border). A small blue square is located to the right of the 'Identifier' field.

Modeling Language



Modeling Language

Circuit Primitives



Data Types

Boundary Elements

Modeling Elements

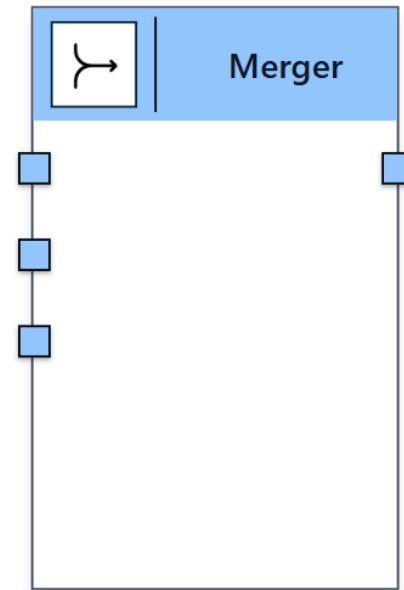
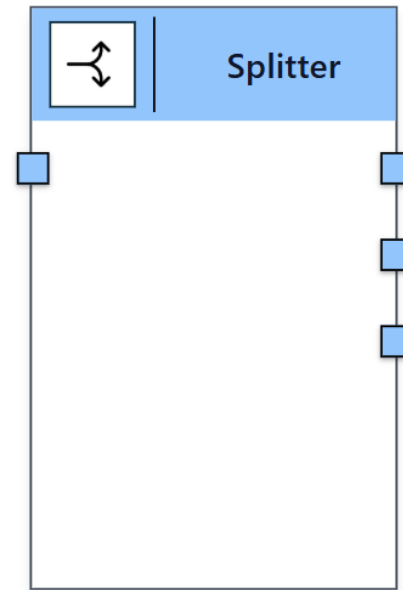
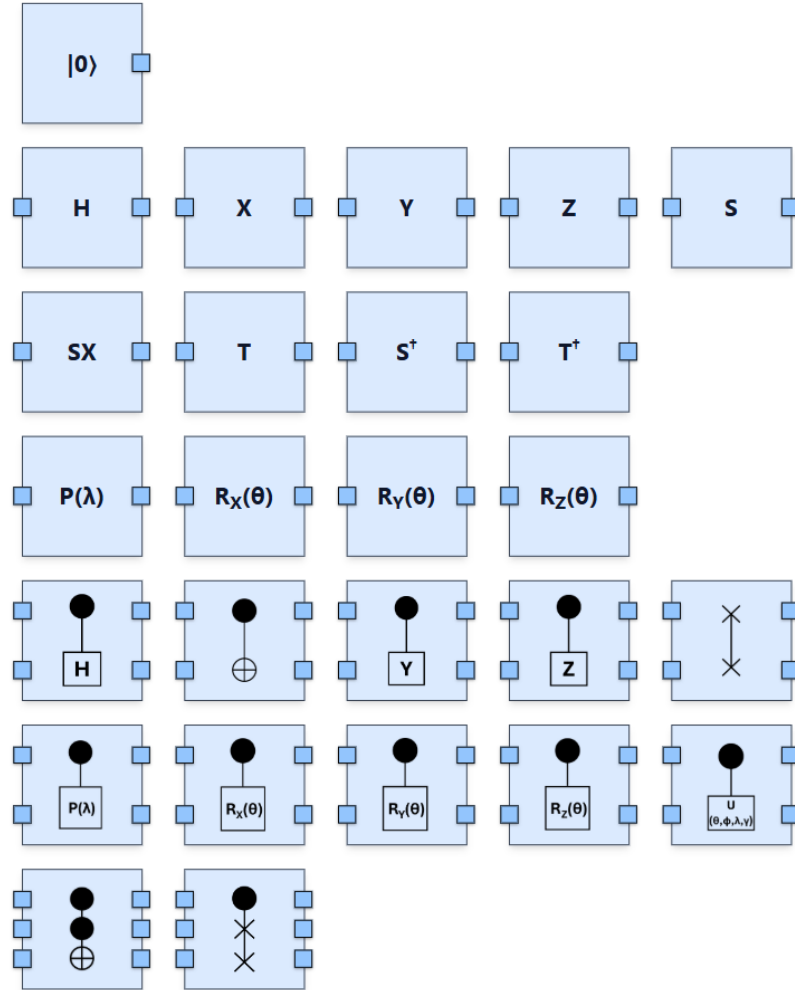
Operators

Control Flow Elements

Custom Elements

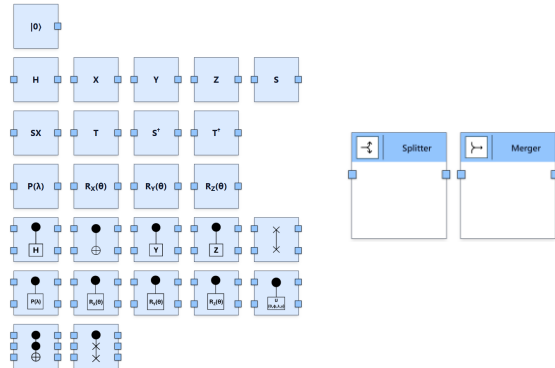
Modeling Language

■ Circuit Primitives:



Modeling Language

Circuit Primitives



Data Types

Boundary Elements

Modeling Elements

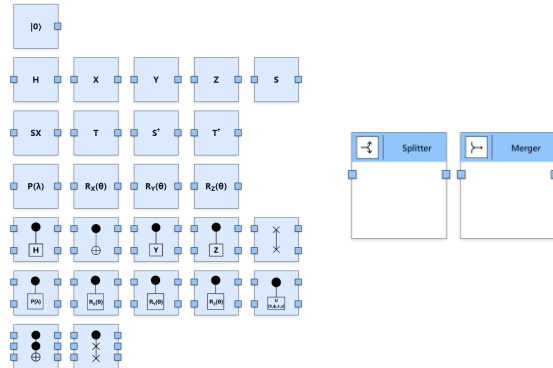
Operators

Control Flow Elements

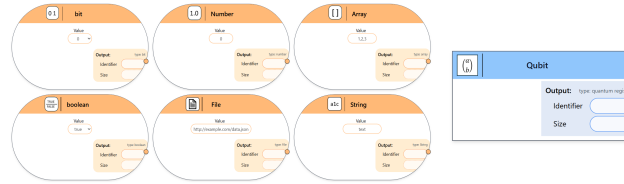
Custom Elements

Modeling Language

Circuit Primitives



Data Types



Boundary Elements

Modeling Elements

Operators

Control Flow Elements

Custom Elements

Modeling Language

■ Data Types:

0 1

bit

Value
0 ▾

Output: type: bit
Identifier
Size

1.0

Number

Value
0

Output: type: number
Identifier
Size

[]

Array

Value
1,2,3

Output: type: array
Identifier
Size

TRUE
FALSE

boolean

Value
true ▾

Output: type: boolean
Identifier
Size



File

Value
<http://example.com/data.json>

Output: type: File
Identifier
Size

a1c

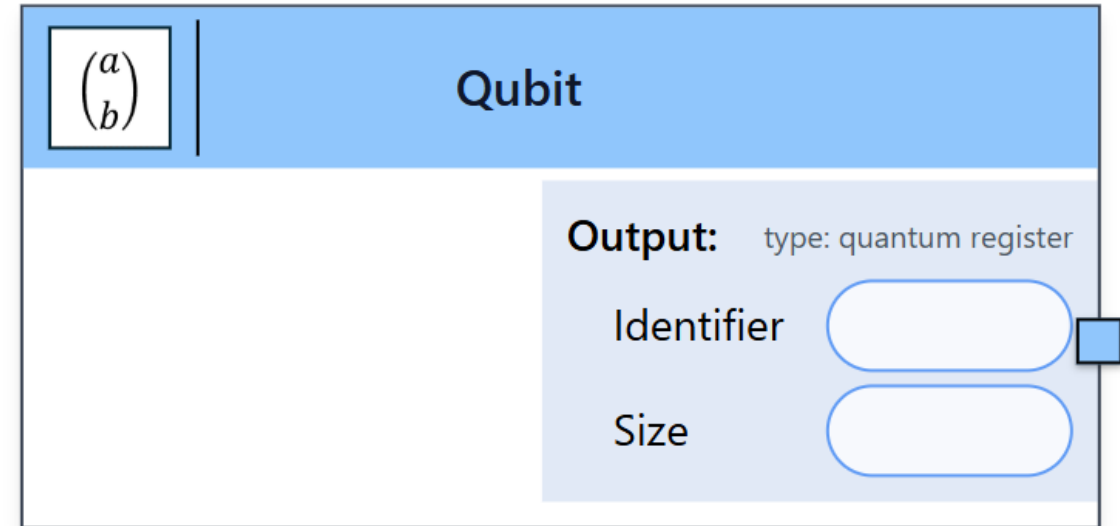
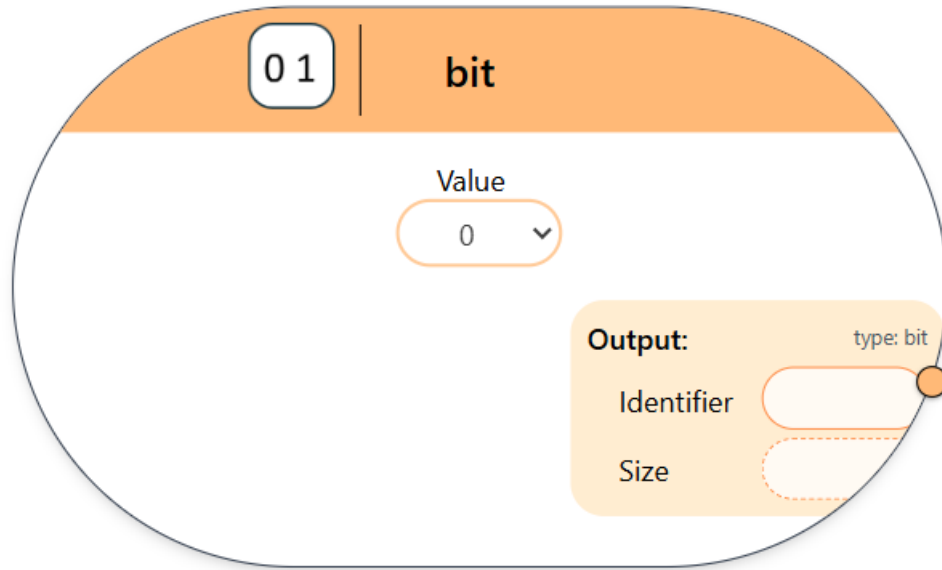
String

Value
text

Output: type: String
Identifier
Size

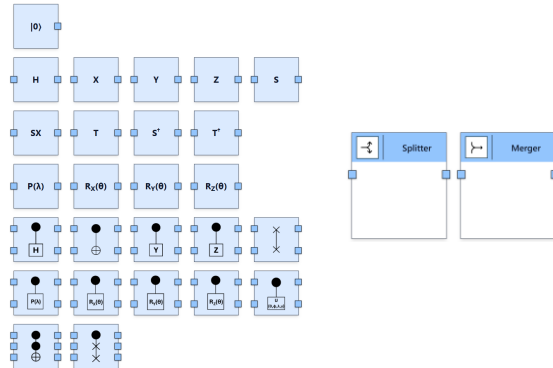
Modeling Language

- Data Types:

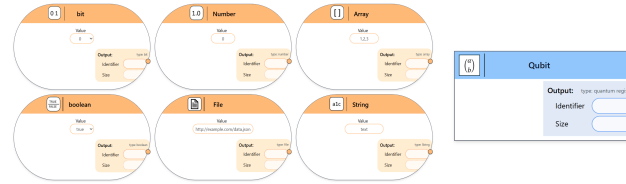


Modeling Language

Circuit Primitives



Data Types



Boundary Elements

Modeling Elements

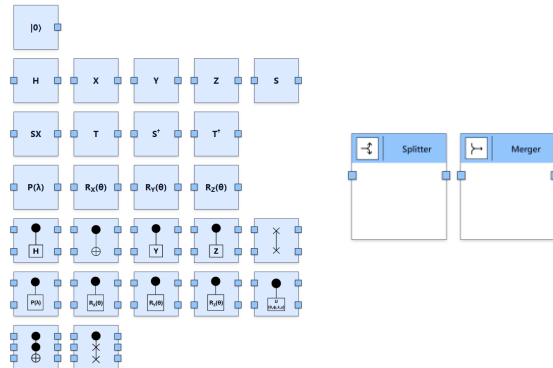
Operators

Control Flow Elements

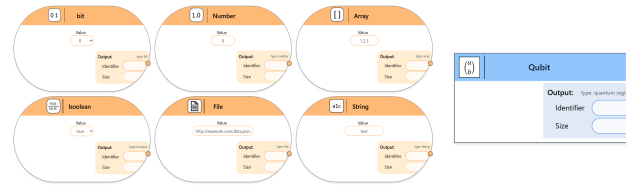
Custom Elements

Modeling Language

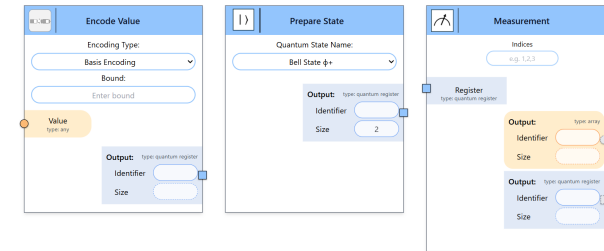
Circuit Primitives



Data Types



Boundary Elements



Modeling Elements

Operators

Control Flow Elements

Custom Elements

Modeling Language

■ Boundary Elements:



Encode Value

Encoding Type:
Basis Encoding ▼

Bound:
Enter bound

Value
type: any

Output: type: quantum register
Identifier
Size



Prepare State

Quantum State Name:
Bell State $\phi+$ ▼

Output: type: quantum register
Identifier
Size 2



Measurement

Indices
e.g. 1,2,3

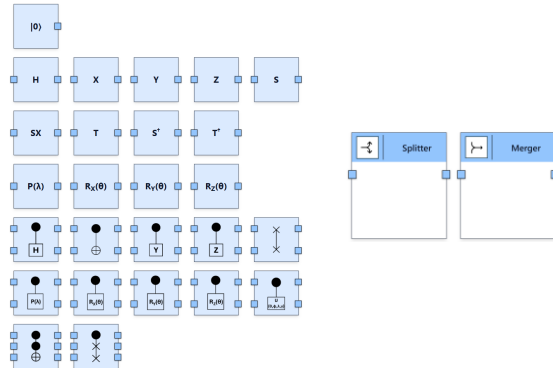
Register
type: quantum register

Output: type: array
Identifier
Size

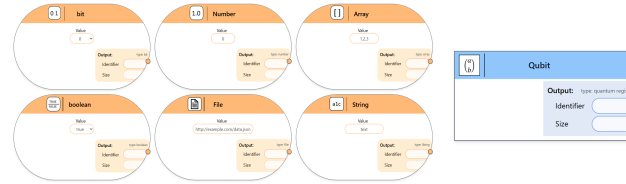
Output: type: quantum register
Identifier
Size

Modeling Language

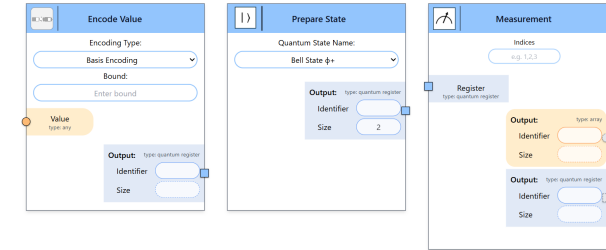
Circuit Primitives



Data Types



Boundary Elements



Modeling Elements

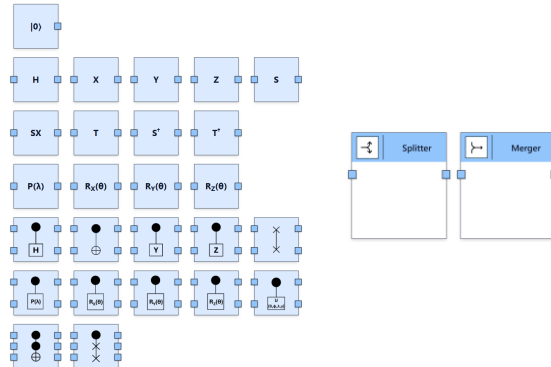
Operators

Control Flow Elements

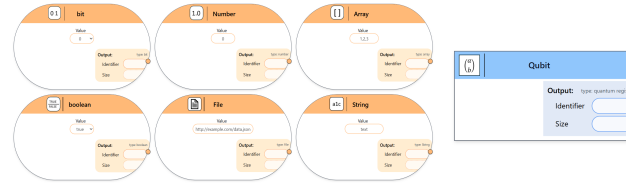
Custom Elements

Modeling Language

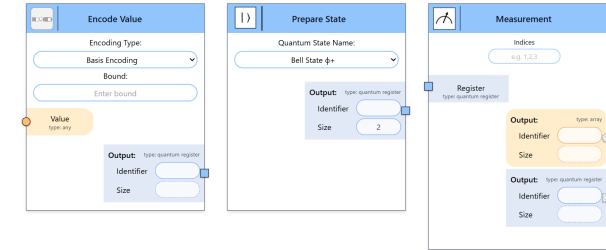
Circuit Primitives



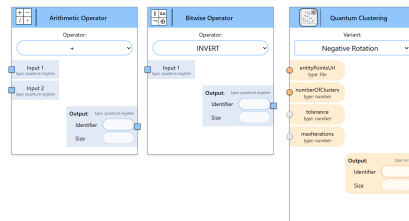
Data Types



Boundary Elements



Modeling Elements



Operators

Control Flow Elements

Custom Elements

■ Operatoren:

+

-

/

*

Arithmetic Operator

Operator:

+

▼

Input 1

type: quantum register

Input 2

type: quantum register

Output:

type: quantum register

Identifier

Size

||

&&

¬

⊕

Bitwise Operator

Operator:

INVERT

▼

Input 1

type: quantum register

Output:

type: quantum register

Identifier

Size

⚙️

⚙️

⚙️

Quantum Clustering

Variant:

Negative Rotation

▼

entityPointsUrl

type: File

numberOfClusters

type: number

tolerance

type: number

maxIterations

type: number

...

Output:

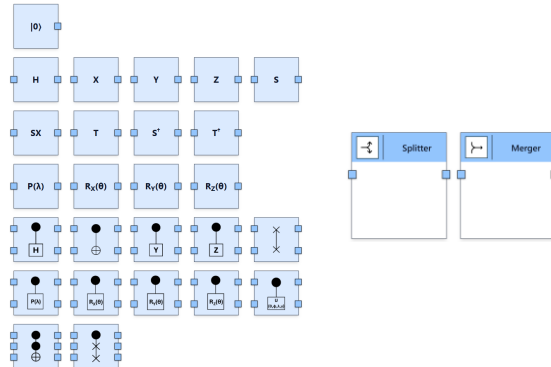
type: array

Identifier

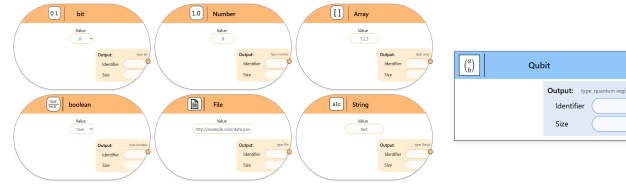
Size

Modeling Language

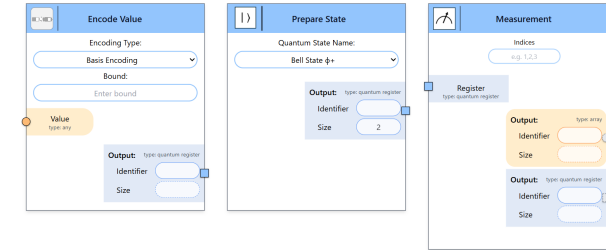
Circuit Primitives



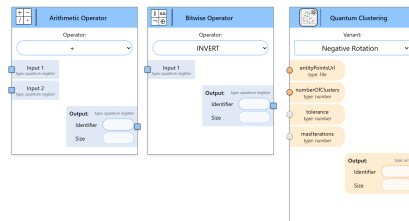
Data Types



Boundary Elements



Modeling Elements



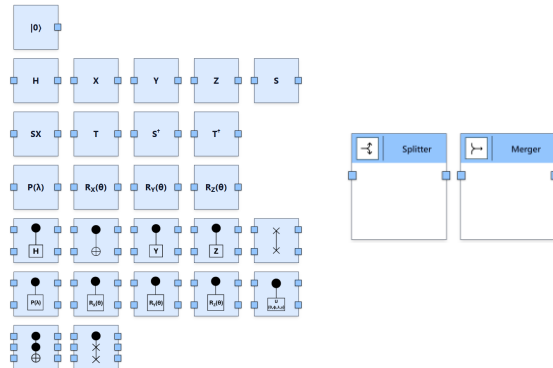
Operators

Control Flow Elements

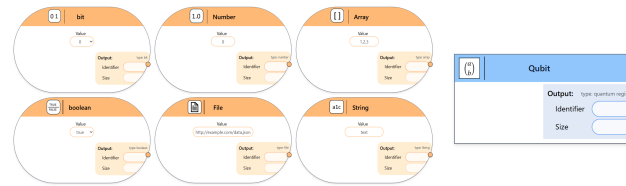
Custom Elements

Modeling Language

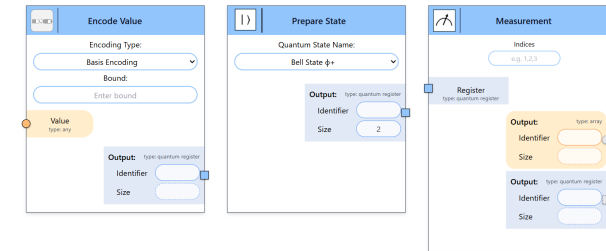
Circuit Primitives



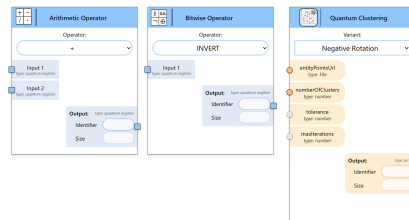
Data Types



Boundary Elements

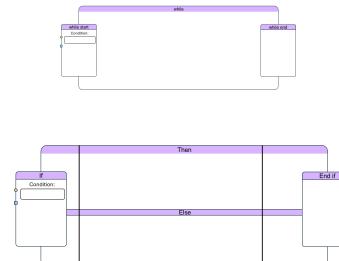


Modeling Elements



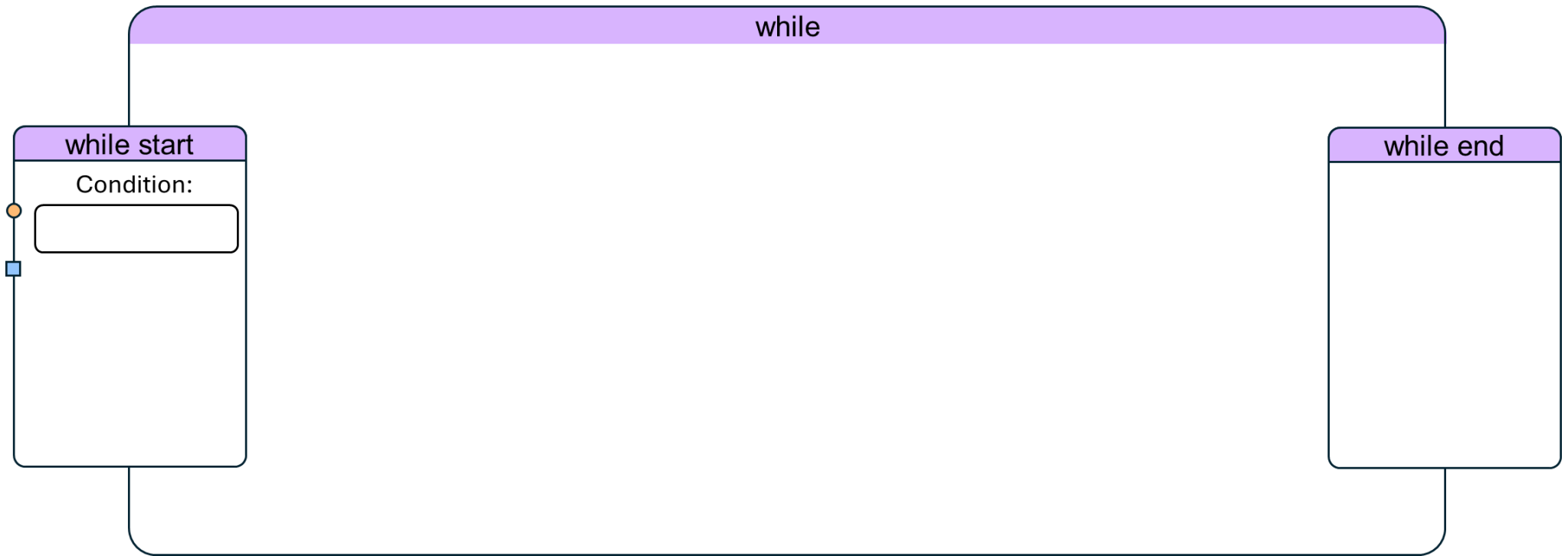
Operators

Control Flow Elements

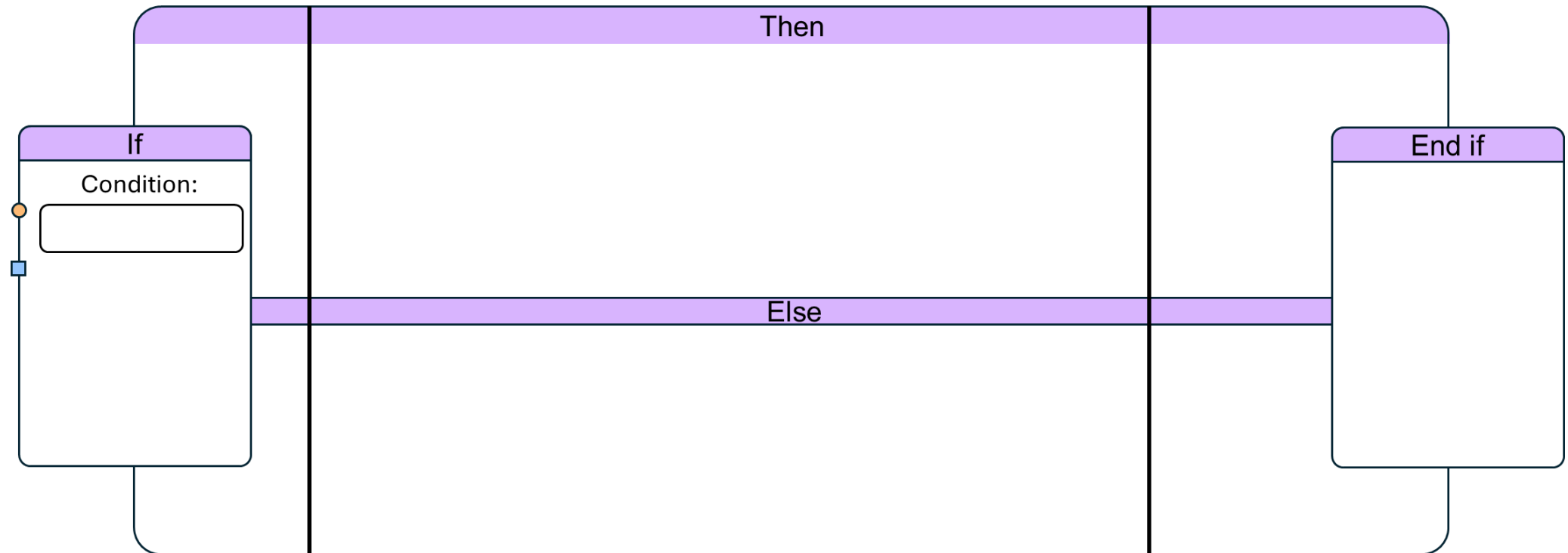


Custom Elements

- Control Flow Elements:

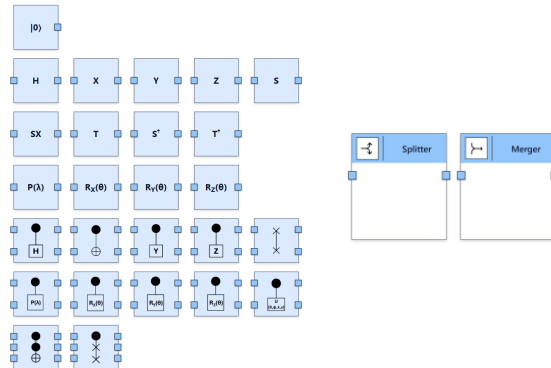


- Control Flow Elements:

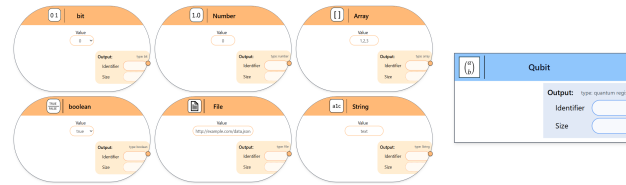


Modeling Language

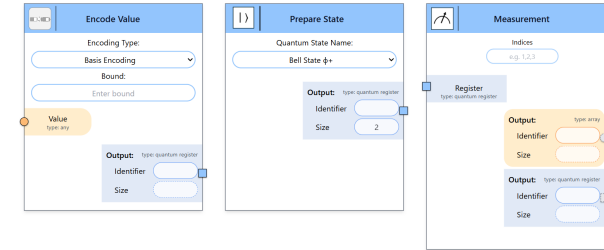
Circuit Primitives



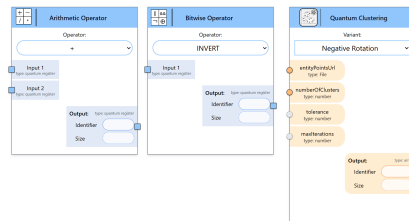
Data Types



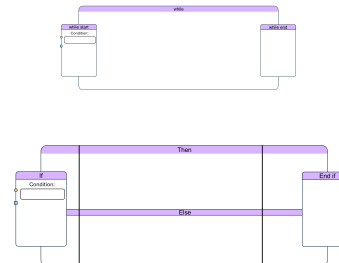
Boundary Elements



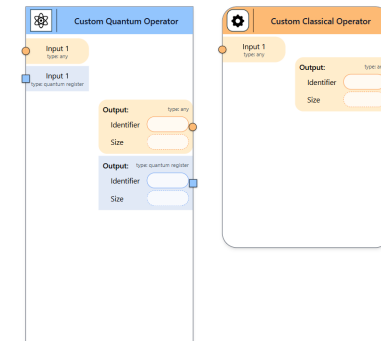
Modeling Elements



Operators

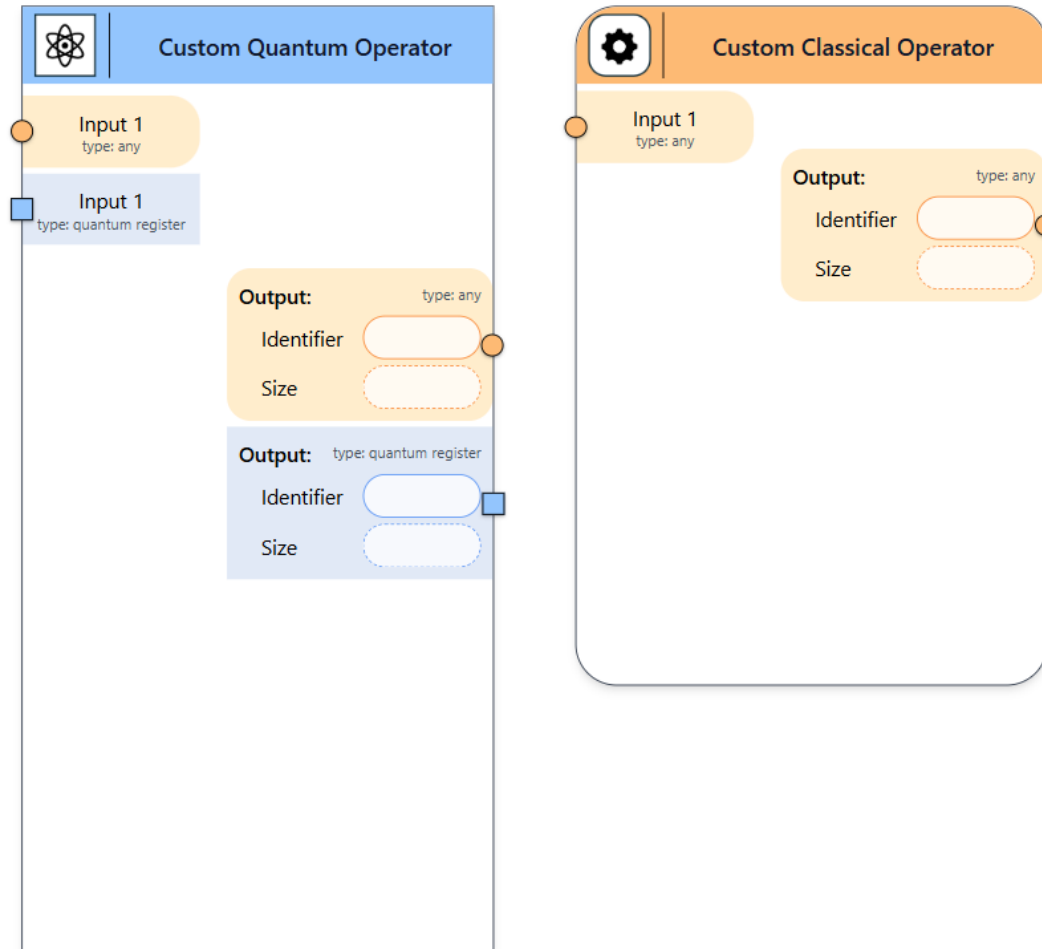


Control Flow Elements

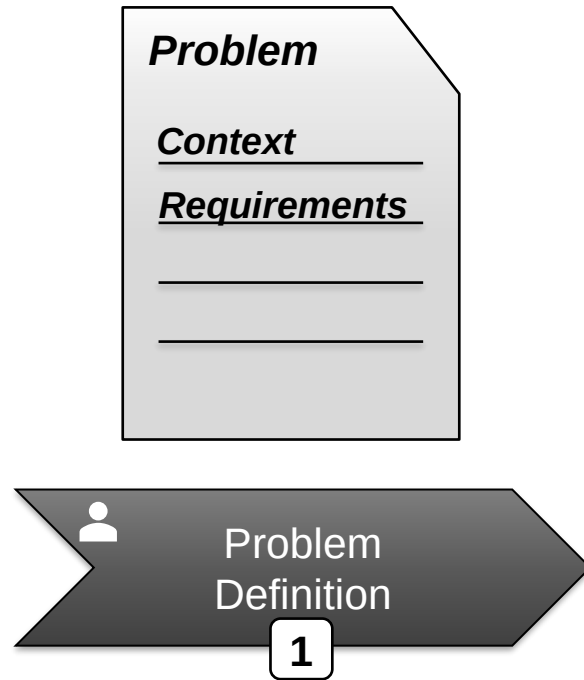


Custom Elements

- Custom Elements:



Modeling and Execution of Quantum Algorithms



Legend:



Manual
Phase



Automated
Phase



Algorithm



Quantum



Implementation

Modeling and Execution of Quantum Algorithms

Problem

I want to cluster my
music data that is
available at this URL:
url.



Problem
Definition

1

Legend:



Manual
Phase



Automated
Phase



Algorithm



Quantum



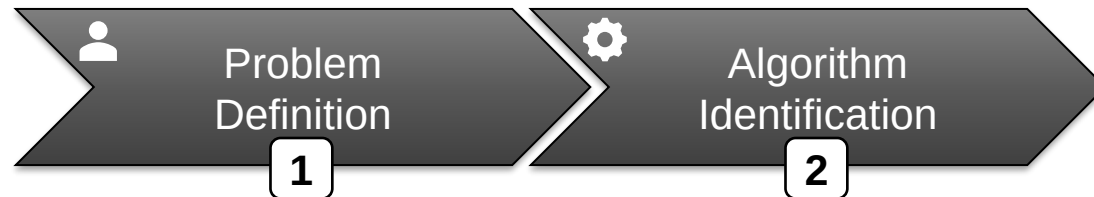
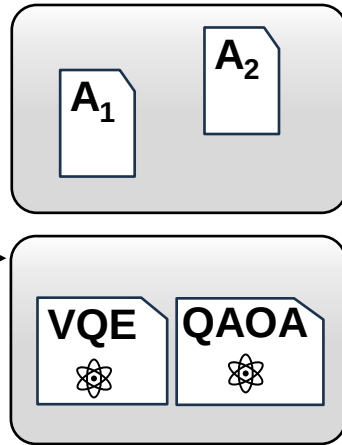
Implementation

Modeling and Execution of Quantum Algorithms

Problem

I have five locations connected by roads.
I want to split the locations into two groups so that as many roads as possible go between the groups.

LLM
→



Legend:



Manual Phase



Automated Phase



Algorithm



Quantum



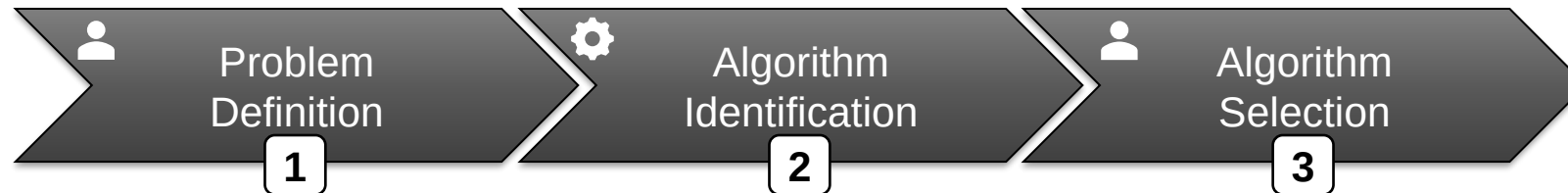
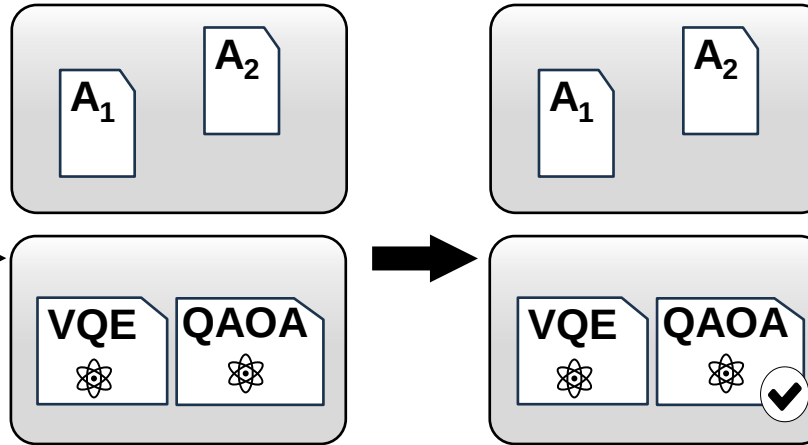
Implementation

Modeling and Execution of Quantum Algorithms

Problem

I have five locations connected by roads.
I want to split the locations into two groups so that as many roads as possible go between the groups.

LLM



Legend:



Manual Phase



Automated Phase



Algorithm



Quantum



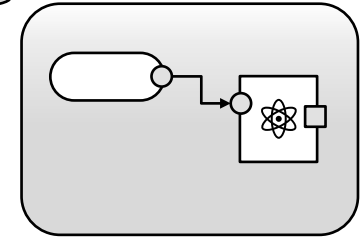
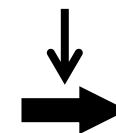
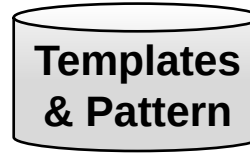
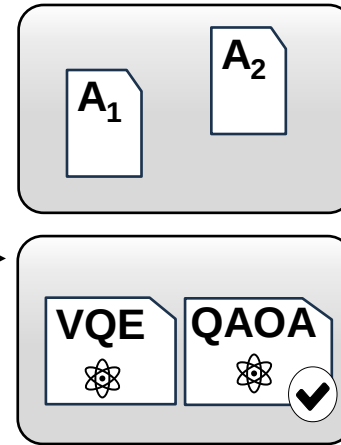
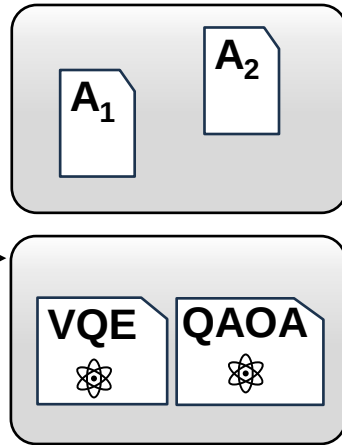
Implementation

Modeling and Execution of Quantum Algorithms

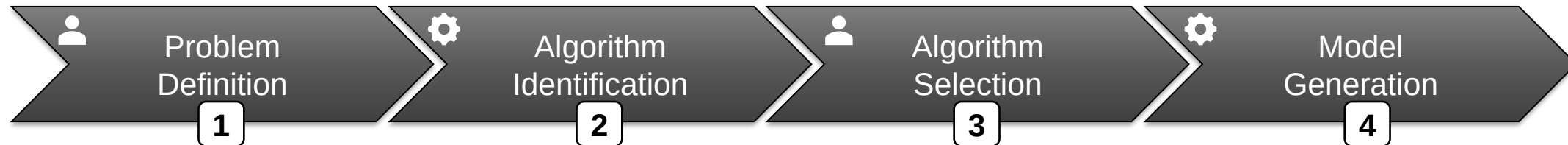
Problem

I have five locations connected by roads.
I want to split the locations into two groups so that as many roads as possible go between the groups.

LLM



Low-Code Model



Legend:



Manual Phase



Automated Phase



Algorithm

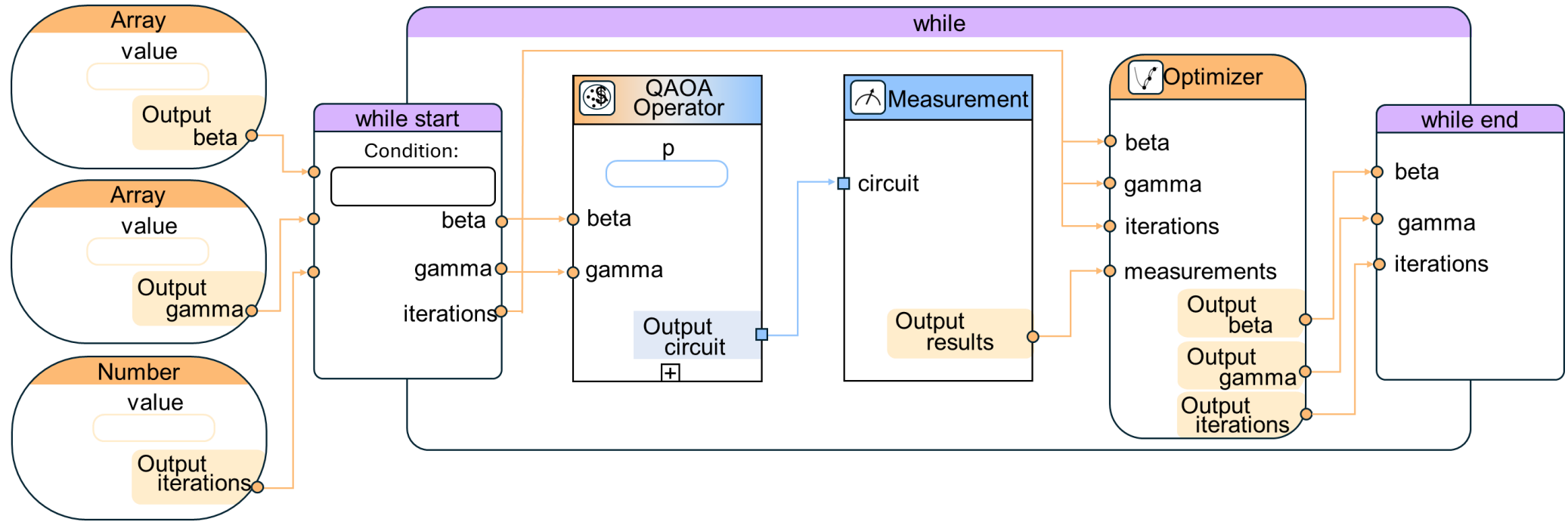


Quantum



Implementation

Modeling and Execution of Quantum Algorithms



Legend:



Classical block



Control flow block



Hybrid block

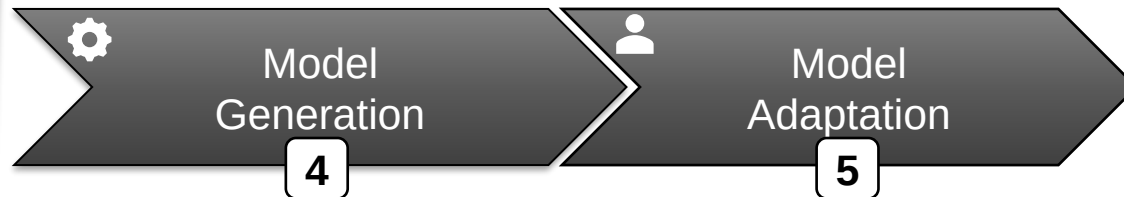
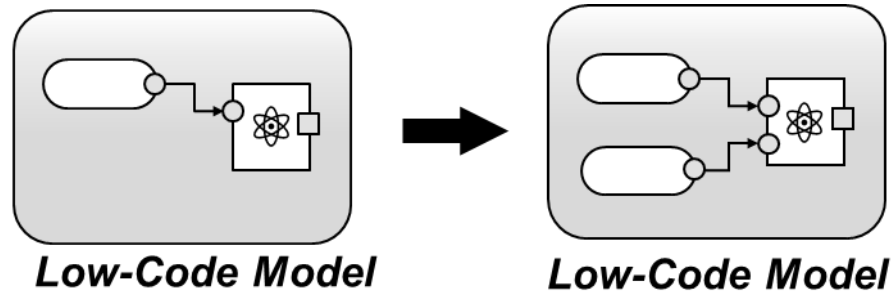


$\oplus$ collapsed



Quantum block

Modeling and Execution of Quantum Algorithms



Legend:



Manual
Phase



Automated
Phase



Algorithm

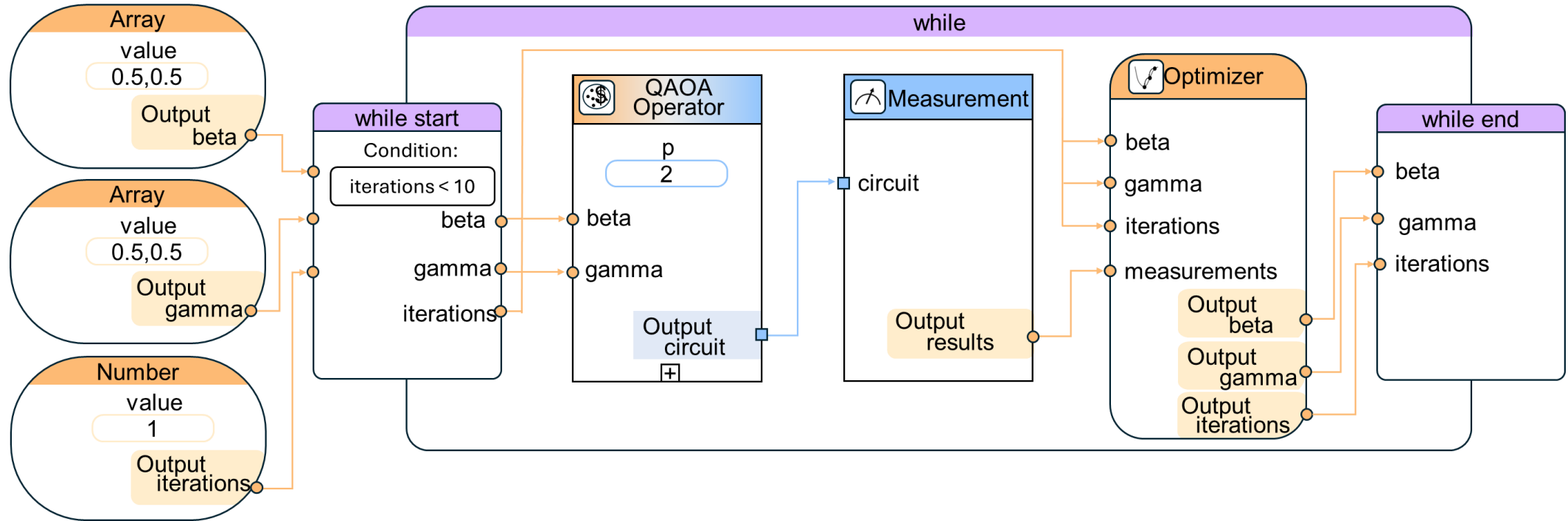


Quantum



Implementation

Modeling and Execution of Quantum Algorithms



Legend:



Classical block



Control flow block



Hybrid block

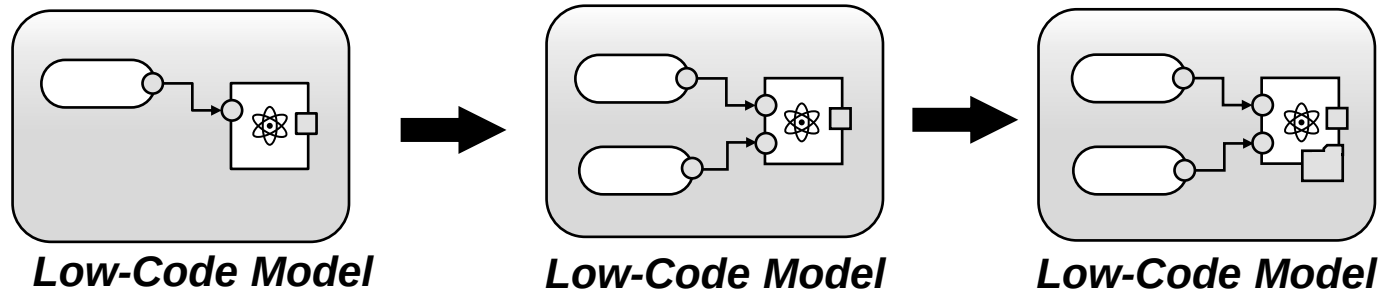


collapsed



Quantum block

Modeling and Execution of Quantum Algorithms



Legend:



Manual
Phase



Automated
Phase



Algorithm

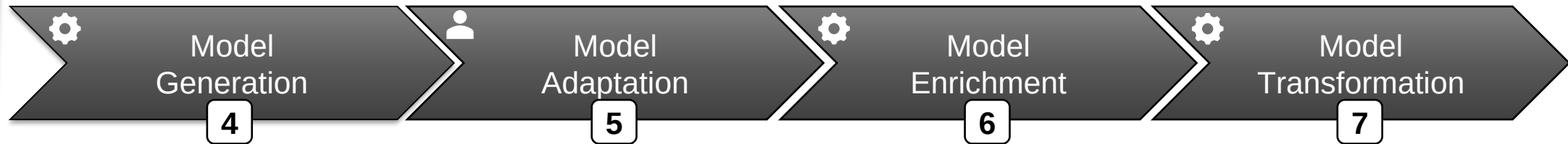
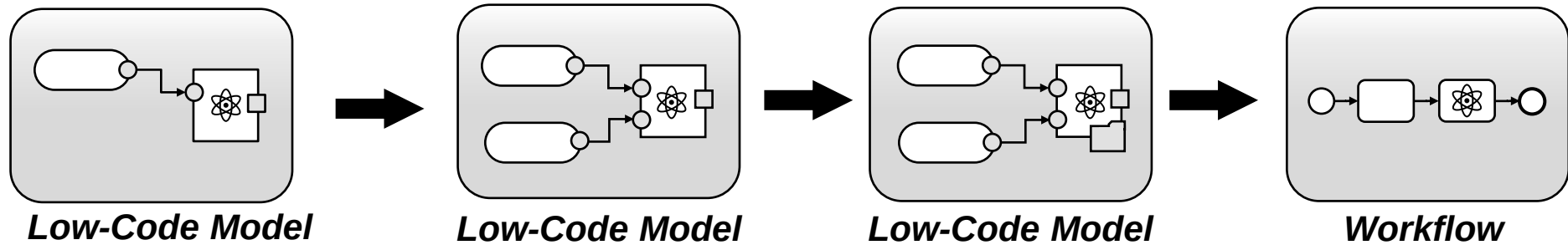


Quantum



Implementation

Modeling and Execution of Quantum Algorithms



Legend:



Manual
Phase



Automated
Phase



Algorithm

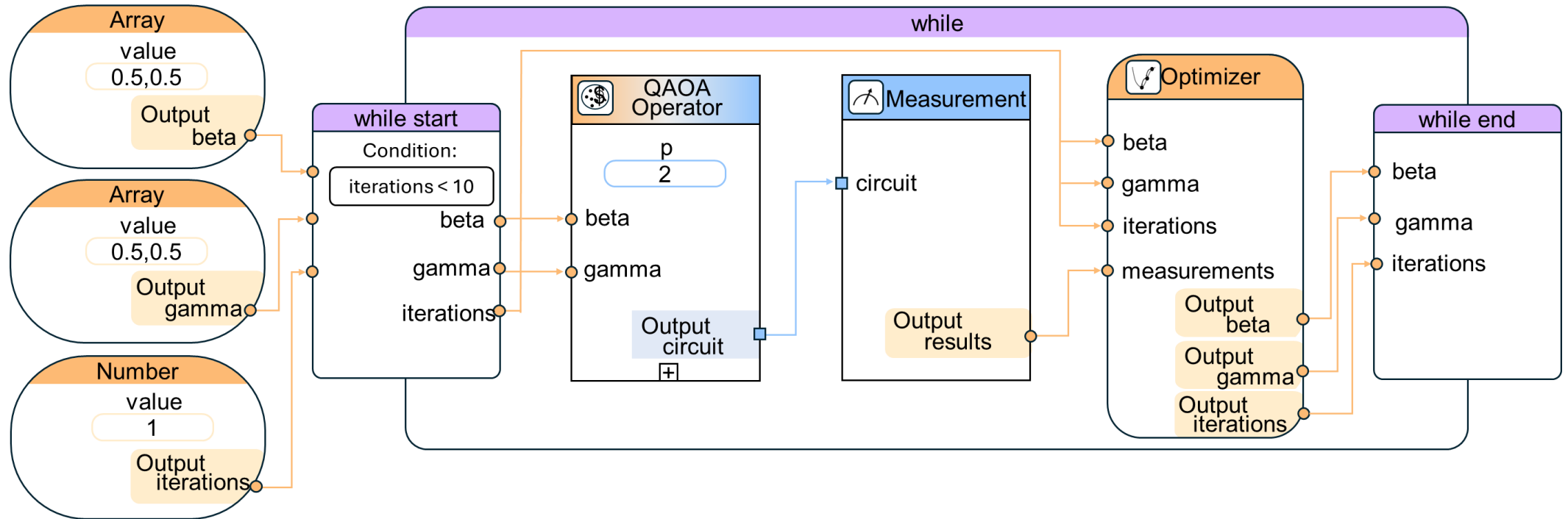


Quantum

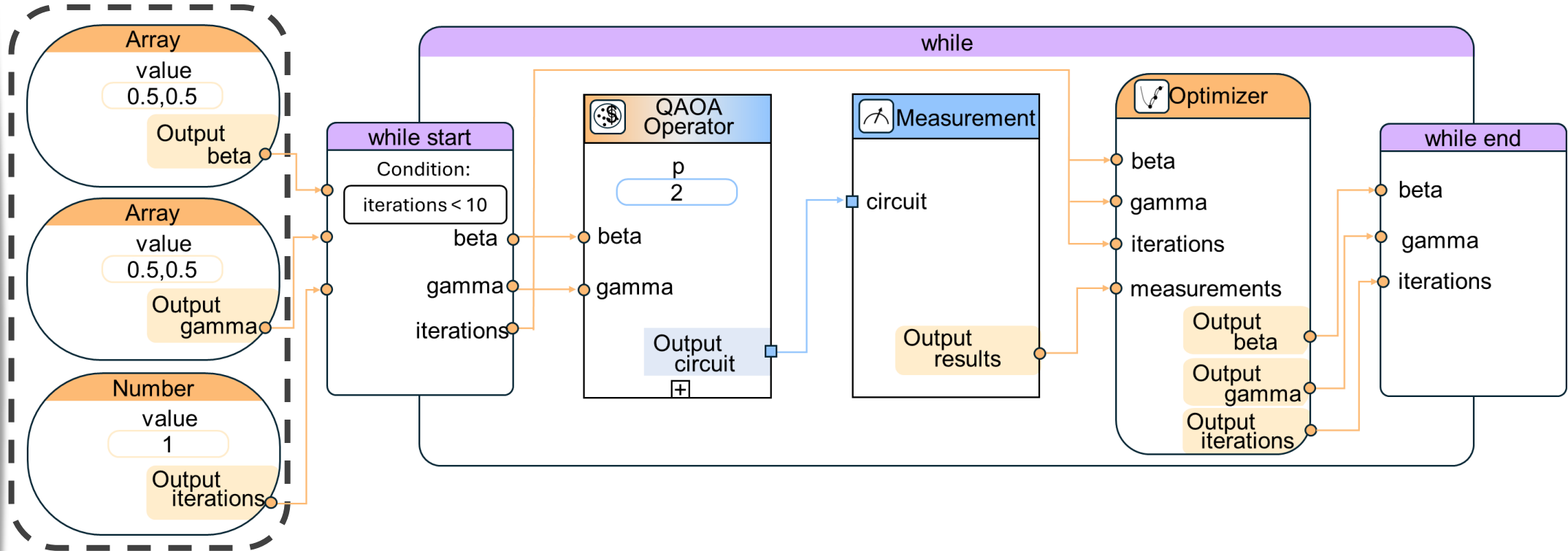


Implementation

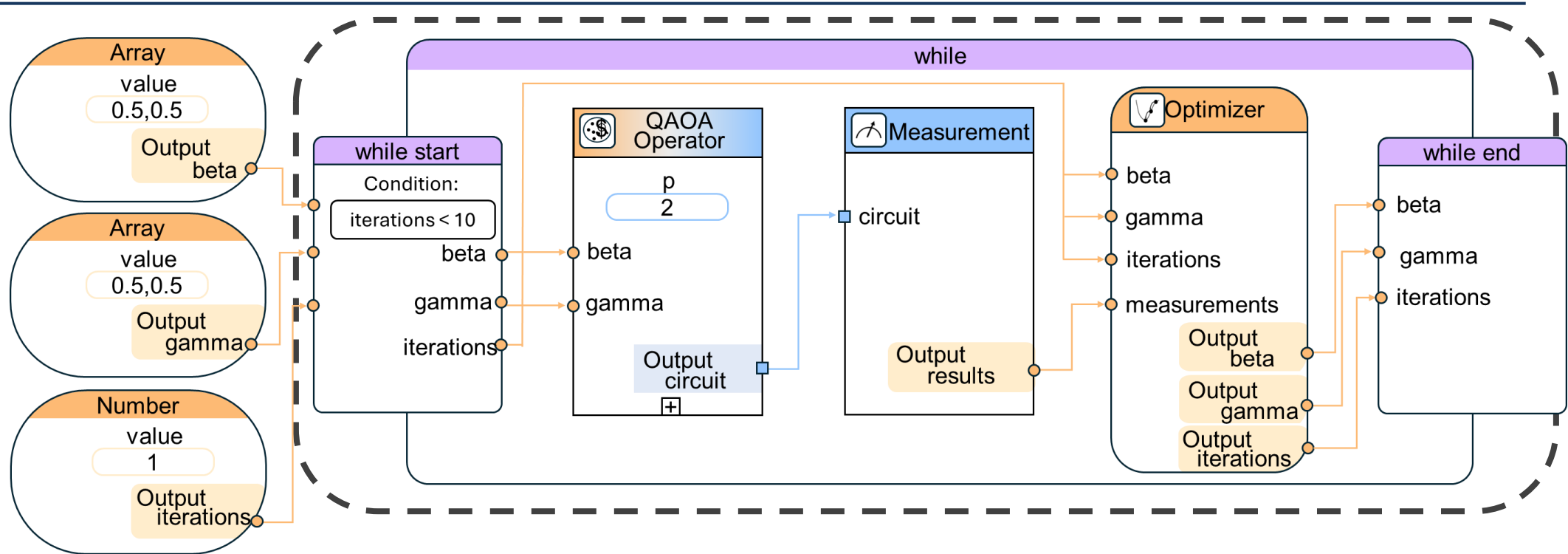
Modeling and Execution of Quantum Algorithms



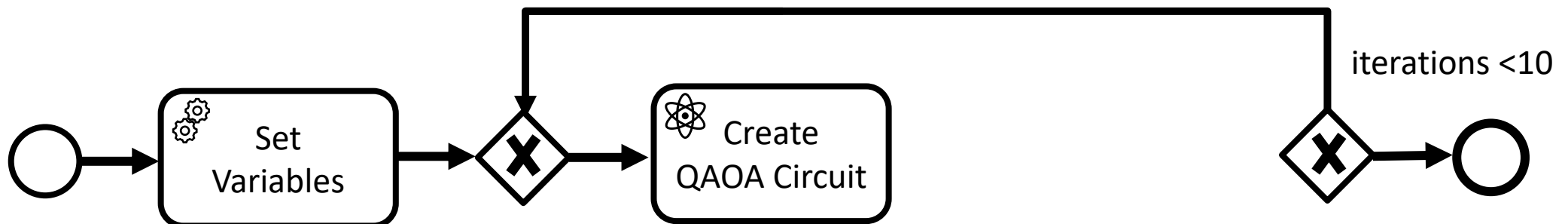
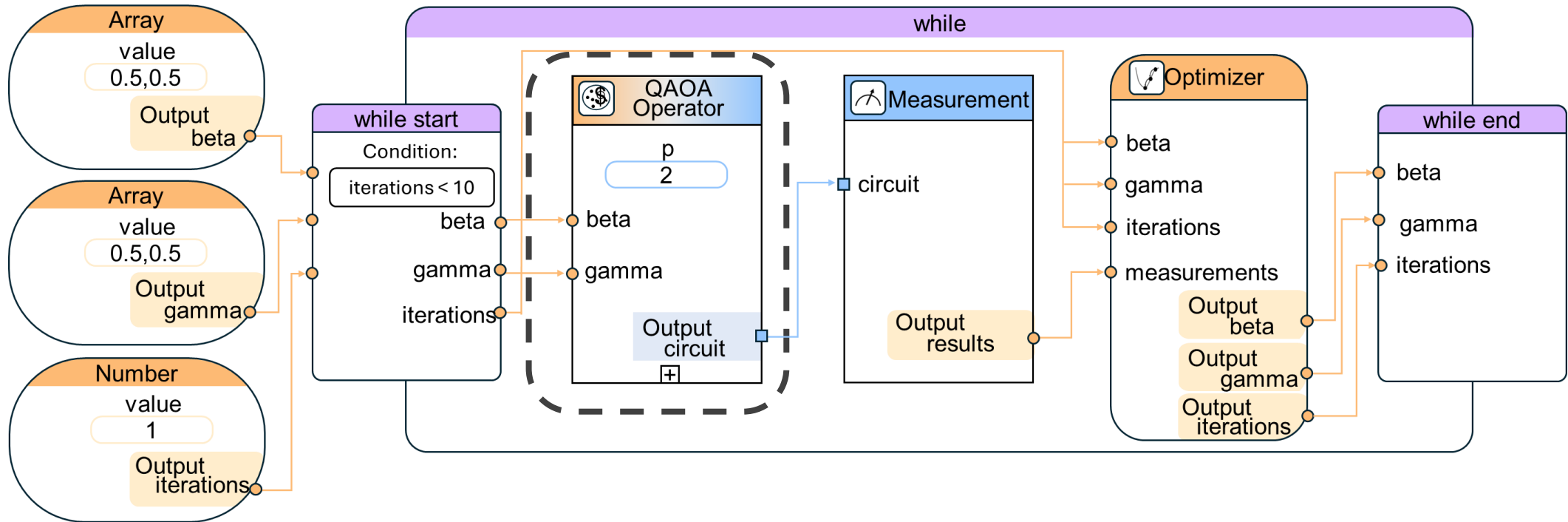
Modeling and Execution of Quantum Algorithms



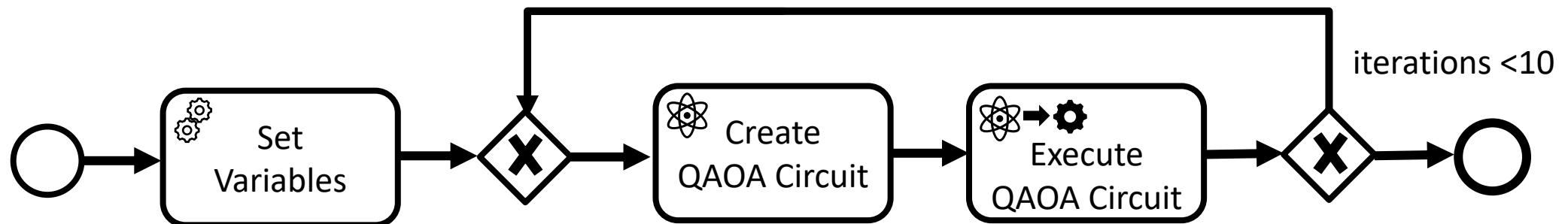
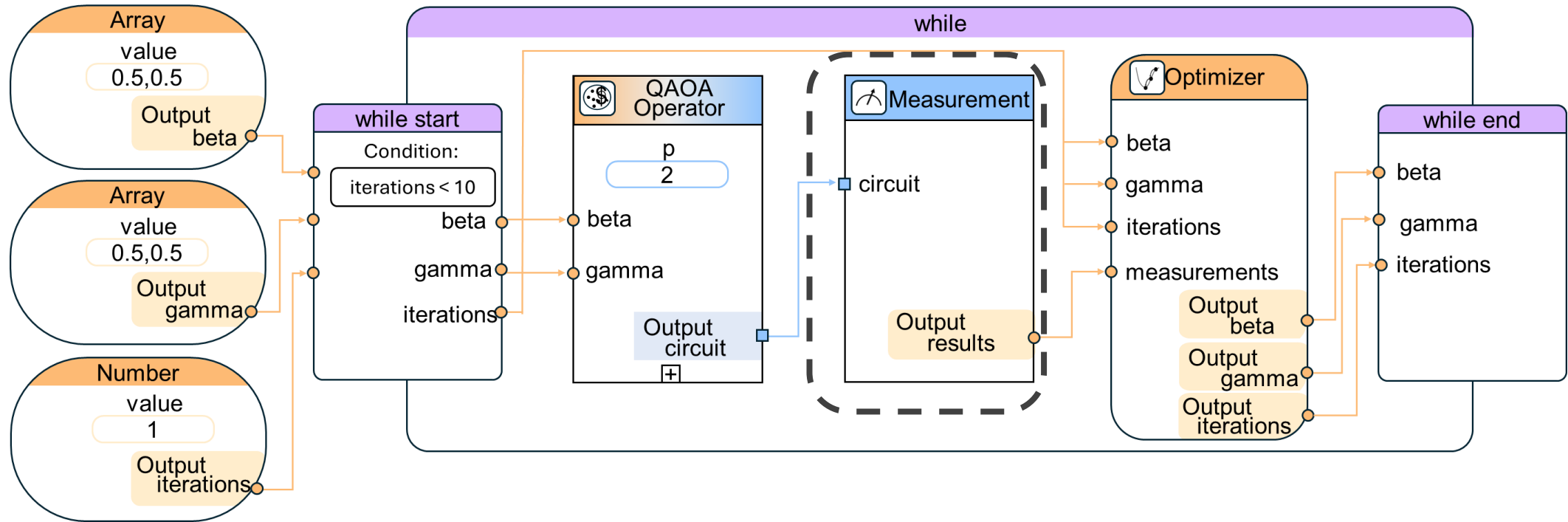
Modeling and Execution of Quantum Algorithms



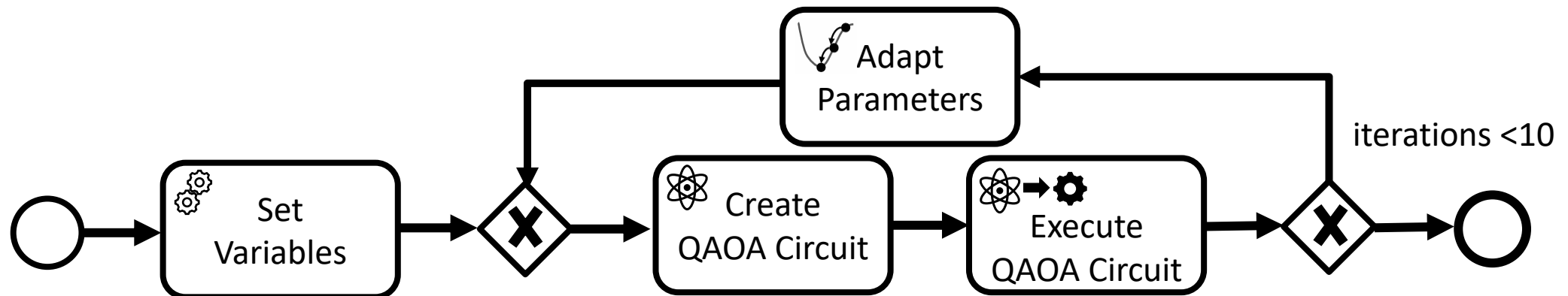
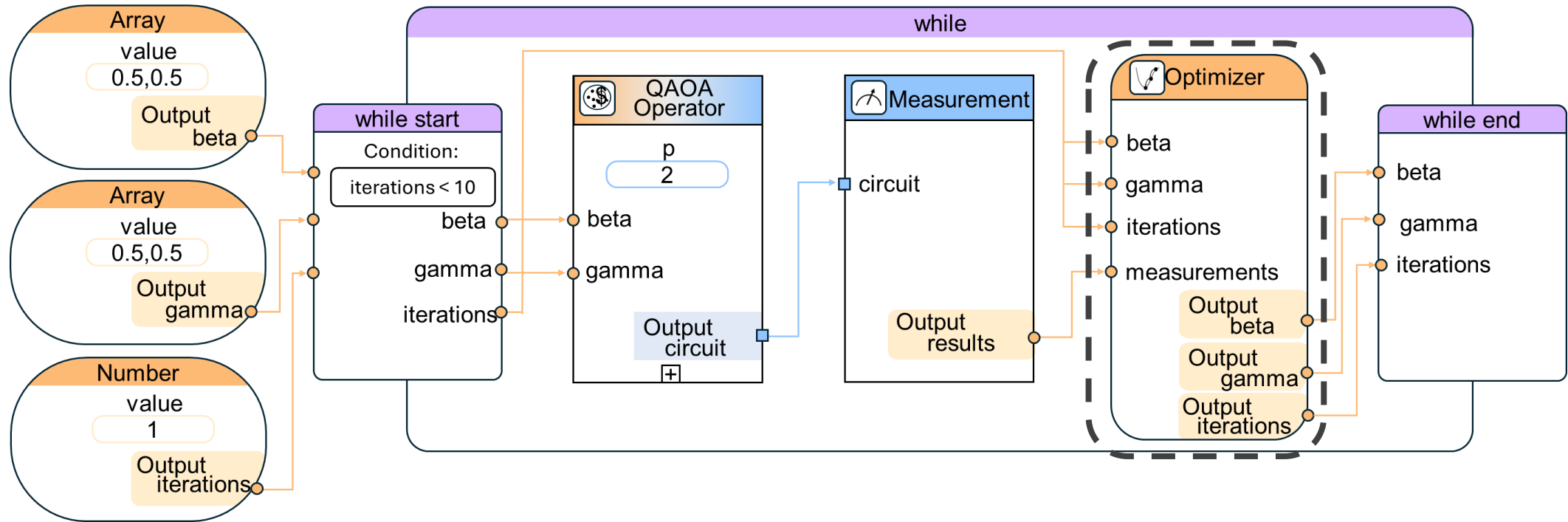
Modeling and Execution of Quantum Algorithms



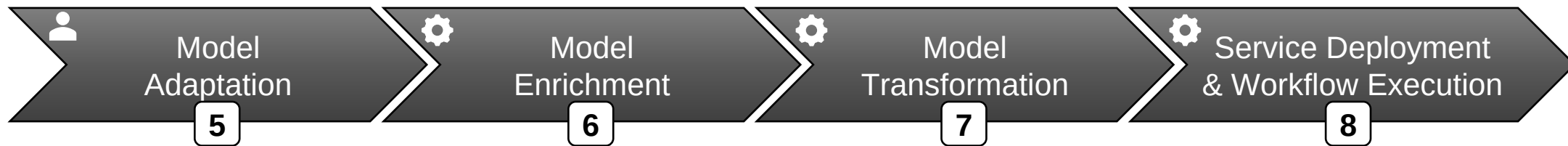
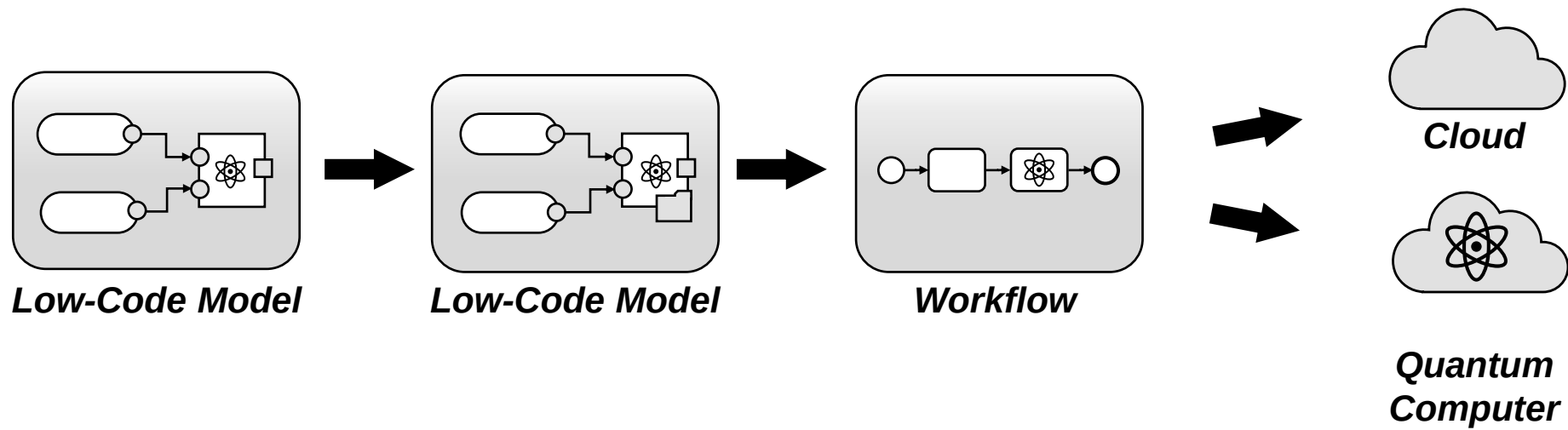
Modeling and Execution of Quantum Algorithms



Modeling and Execution of Quantum Algorithms



Modeling and Execution of Quantum Algorithms



Legend:



Manual
Phase



Automated
Phase



Algorithm



Quantum



Implementation

Limitations

- LLM recommendations may be inconsistent or hallucinate
- Quality depends on available templates and pattern coverage
- Missing implementations may require manual OpenQASM coding

Conclusion

- Introduced a low-code modeling language for quantum applications
- Combined:
 - Low-code modeling
 - Workflows
 - LLM-assisted algorithm recommendation
- Aims to reduce the entry barrier for non-experts in quantum software development

Thank you for your attention!